香港中文大學
The Chinese University of Hong Kong

ROSE5710 Fundamental Robotics and Systems

Lecture Set 4: Position Forward Kinematics

Darwin Lau

2026-27

Lecture Summary

Aims of the lecture

- To formulate the FK using transformation matrices
- To formulate the FK using D-H parameters

References

- Robotics, Vision and Control, 3rd edition, Peter Corke, Springer, 2023
- Introduction to Robotics: Mechanics and Control, 3rd edition, J. Craig's, Pearson, 2005

What is Robot Kinematics?

Kinematics refer to the study of motion of objects only

- Do not consider the forces
- So this involves the study of position, velocity, acceleration etc.

Two spaces for robot kinematics

- Generalised coordinates **(joint space)**
- End-effector coordinates **(operational or task space)**



- Start with position level

What is Robot Kinematics?

It will be shown that the following relationship naturally forms

$$\begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix} = \begin{bmatrix} f_1(q_1,\ldots,q_n) \\ \vdots \\ f_p(q_1,\ldots,q_n) \end{bmatrix} \qquad \mathbf{x} = \mathbf{f}(\mathbf{q})$$



Forward kinematics (FK)

- Useful when processing feedback from joint sensors



Inverse kinematics (IK)

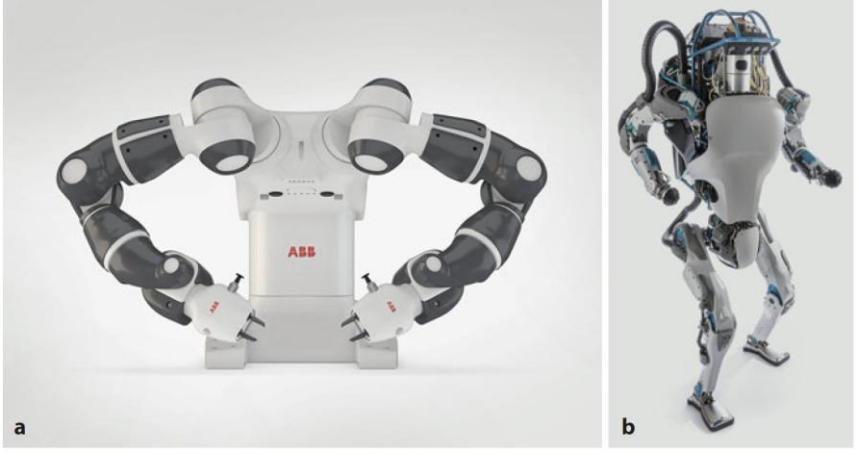
- Useful when commanding the robot to produce a certain task



What is Robot Kinematics?

Real-world examples





Simplest Example

The FK is the “easier” (compared with IK) and certainly a basic problem

- Because the generalised coordinates are the unique variables

For example

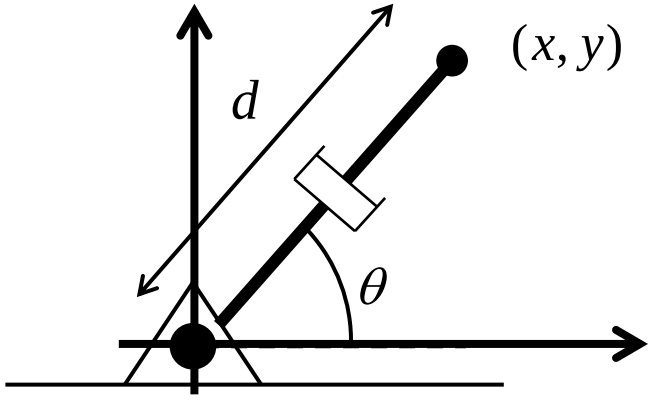
- What is the joint space?

$$\mathbf{q} = \begin{bmatrix} \theta \\ d \end{bmatrix}$$

- What is the task space?

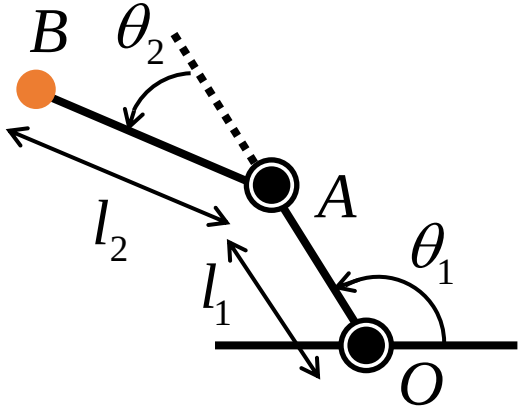
$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

- How are they related?



$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} d\cos\theta \\ d\sin\theta \end{bmatrix} \Rightarrow \mathbf{x} = \begin{bmatrix} f_1(\mathbf{q}) \\ f_2(\mathbf{q}) \end{bmatrix} = \mathbf{f}(\mathbf{q})$$

2R Planar Robot



Similarly

- What is the joint space?

$$\mathbf{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

- What is the task space?

$$\mathbf{x} = \begin{bmatrix} x_B \\ y_B \end{bmatrix}$$

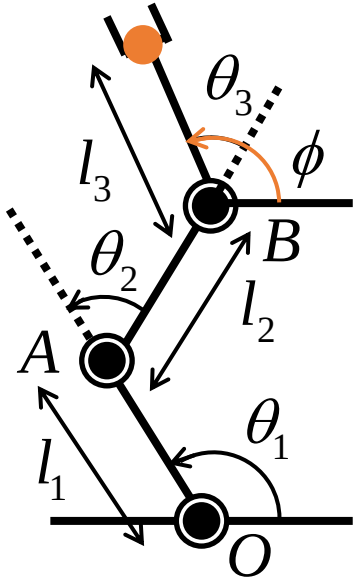
- How are they related?

$${}^0\mathbf{r}_{OB} = {}^0\mathbf{r}_{OA} + {}^0\mathbf{r}_{AB}$$

$$= \begin{bmatrix} l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2) \\ l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2) \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} f_1(\mathbf{q}) \\ f_2(\mathbf{q}) \end{bmatrix} = \mathbf{f}(\mathbf{q})$$

3R Planar Robot



Similarly

- What is the joint space?

$$\mathbf{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

- What is the task space?

$$\mathbf{x} = \begin{bmatrix} x_P \\ y_P \\ \phi \end{bmatrix}$$

- How are they related?

$$\begin{bmatrix} x_P \\ y_P \end{bmatrix} = \begin{bmatrix} l_1 c_{\theta_1} + l_2 c_{\theta_1+\theta_2} + l_3 c_{\theta_1+\theta_2+\theta_3} \\ l_1 s_{\theta_1} + l_2 s_{\theta_1+\theta_2} + l_3 s_{\theta_1+\theta_2+\theta_3} \end{bmatrix}$$

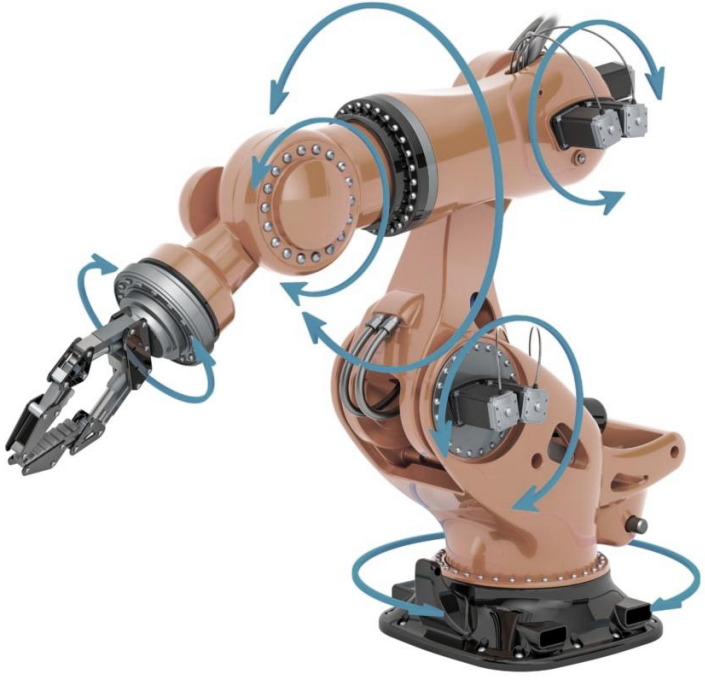
$$\phi = \theta_1 + \theta_2 + \theta_3$$

$$\mathbf{x} = \begin{bmatrix} f_1(\mathbf{q}) \\ f_2(\mathbf{q}) \\ f_3(\mathbf{q}) \end{bmatrix} = \mathbf{f}(\mathbf{q})$$

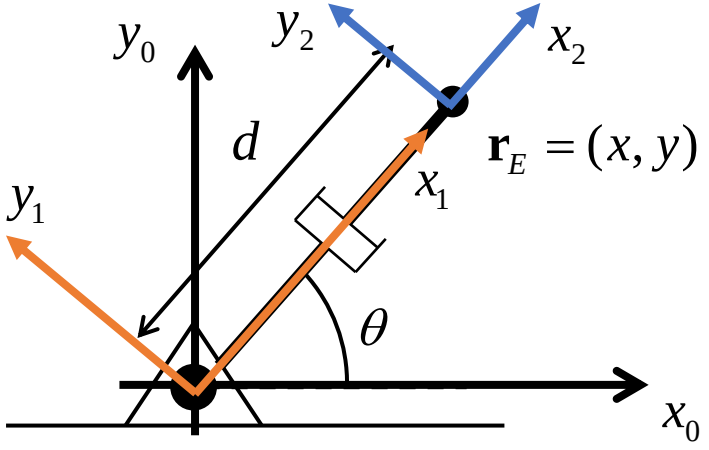
FK using Frames and Transformations

Geometric approach gets challenging in 3D

- Use frames and transformations



Simple Example



Go back to our simplest example

- Step 1: define frames

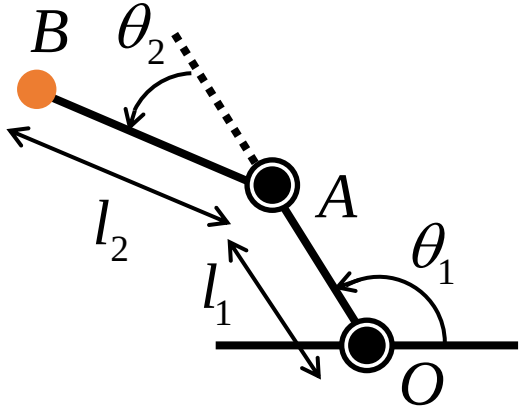
- Step 2: write the homogeneous transforms

$$ {}^0_1T = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad {}^1_2T = \begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

- Step 3: Solve for the FK

$$ {}^0_2T = {}^0_1T\,{}^1_2T = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & d\cos\theta \\ \sin\theta & \cos\theta & 0 & d\sin\theta \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

2R Planar



Go back to our simplest example

- Step 1: define frames

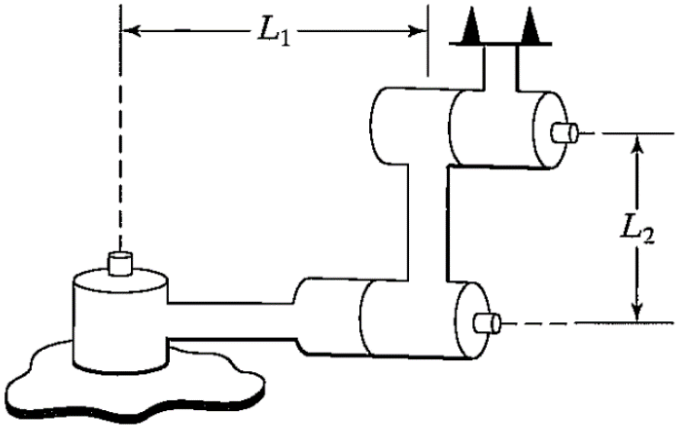
- Step 2: write the homogeneous transforms

$$ {}^0_1T = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 & 0 \\ \sin\theta_1 & \cos\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^1_2T = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & l_1 \\ \sin\theta_2 & \cos\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^2_3T = \begin{bmatrix} 1 & 0 & 0 & l_2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

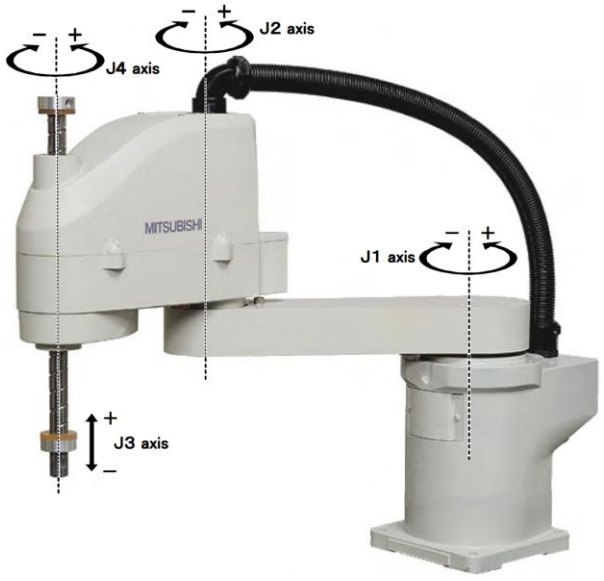
- Step 3: Solve for the FK

$$ {}^0_3T = {}^0_1T\,{}^1_2T\,{}^2_3T = \begin{bmatrix} c_{\theta_1+\theta_2} & -s_{\theta_1+\theta_2} & 0 & l_1c_{\theta_1} + l_2\,c_{\theta_1+\theta_2} \\ s_{\theta_1+\theta_2} & c_{\theta_1+\theta_2} & 0 & l_1s_{\theta_1} + l_2\,s_{\theta_1+\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

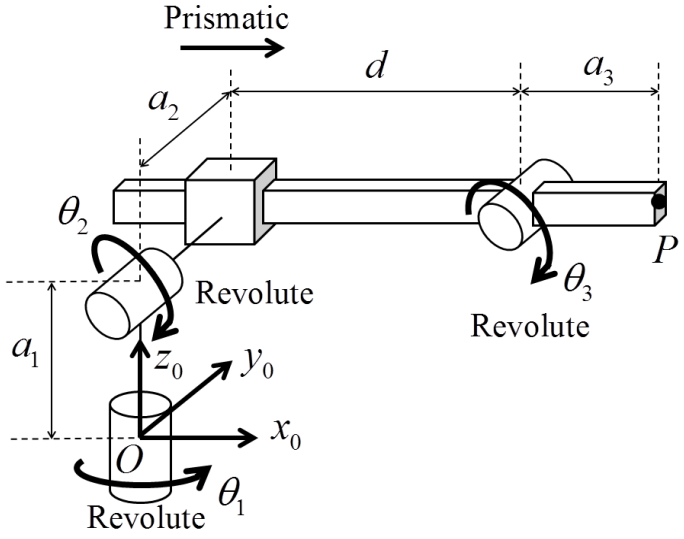
Exercise 1



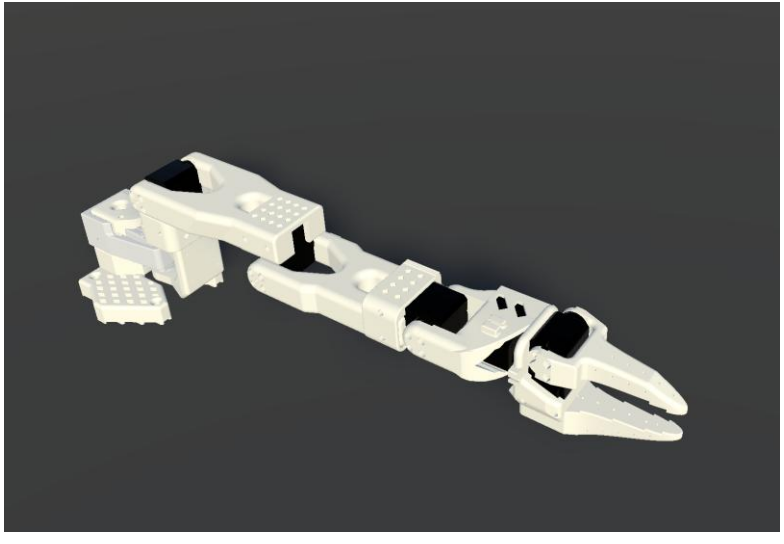
Exercise 2

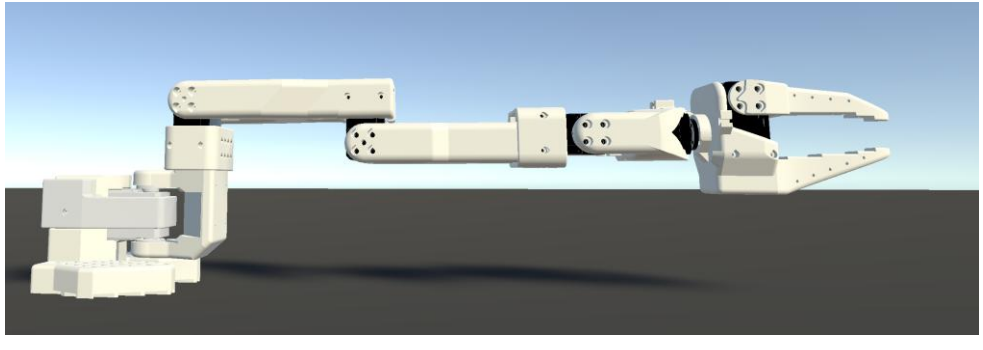


Exercise 3


Prismatic
d
a_3
a_2
θ_2
Revolute
θ_3
Revolute
P
a_1
z_0
y_0
x_0
O
θ_1
Revolute

Exercise 4





Summary

Advantages for this approach

- Systematic and easy to follow
- “Simple” to quickly derive the position and orientation

Disadvantages

- Frames and rotation matrices are arbitrarily assigned by the user
- Therefore, it is not standardised (different valid assignments)
- Difficult to document and share the information to others
- Dimensions of the manipulator are not stored in any way

Note: Even with different frames, the resulting FK would be the same!

Last Tip

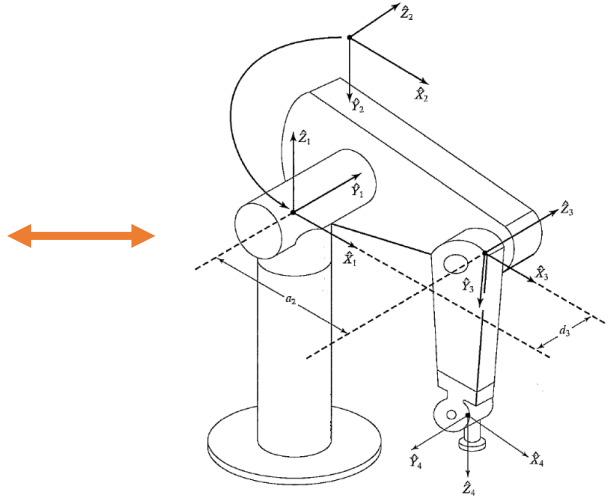
One frame per joint or one frame per link?

Why Denavit-Hartenberg Representation?

A more standardised approach to generate the FK

- Based on assigning frames and variables in a certain way
- Naturally becomes an easy way to document the representation and parameters of a robot manipulator in books and papers

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	0	θ_1
2	-90°	0	0	θ_2
3	0	a_2	d_3	θ_3
4	-90°	a_3	d_4	θ_4
5	90°	0	0	θ_5
6	-90°	0	0	θ_6



香港中文大學
The Chinese University of Hong Kong

Why Denavit-Hartenberg Representation?



UNIVERSAL ROBOTS | Articles | Download | Safety & Security | Forum | myUR | Go to Main Site

Support › Articles › DH Parameters for calculations of kinematics and dynamics

DH PARAMETERS FOR CALCULATIONS OF KINEMATICS AND DYNAMICS

Denavit Hartenberg Parameters - DH Parameters

Last modified on Sep 10, 2025

Denavit–Hartenberg parameters are used to calculate kinematics and dynamics of UR robots.

The definition of the Denavit–Hartenberg parameters can be found here:
http://en.wikipedia.org/wiki/Denavit%E2%80%93Hartenberg_parameters

Attached files

DH-Transformation.xlsx

Related articles

DH Parameters for calculations of kinematics and dynamics UR20

DH Parameters for calculations of kinematics and dynamics UR30

DH Parameters for calculations of kinematics and dynamics UR15

UR10e/UR12e							
Kinematics	a [m]	d [m]	alpha [rad]	**Dynamics**	Mass [kg]	Center of Mass [m]	Inertia Matrix
Joint 1	0	0.1807	π/2	Link 1	7.369	[0.021, 0.000, 0.027]	[0.0341, 0.0000, -0.0043, 0.0000, 0.0353, 0.0001, -0.0043, 0.0001, 0.0216]
Joint 2	-0.6127	0	0	Link 2	13.051	[0.38, 0.000, 0.158]	[0.0281, 0.0001, -0.0156, 0.0001, 0.7707, 0.0000, -0.0156, 0.0000, 0.7694]
Joint 3	-0.57155	0	0	Link 3	3.989	[0.24, 0.000, 0.068]	[0.0101, 0.0001, 0.0092, 0.0001, 0.3093, 0.0000, 0.0092, 0.0000, 0.3065]
Joint 4	0	0.17415	π/2	Link 4	2.1	[0.000, 0.007, 0.018]	[0.0030, -0.0000, -0.0000, -0.0000, 0.0022, -0.0002, -0.0000, -0.0002, 0.0026]
Joint 5	0	0.11985	-π/2	Link 5	1.98	[0.000, 0.007, 0.018]	[0.0030, -0.0000, -0.0000, -0.0000, 0.0022, -0.0002, -0.0000, -0.0002, 0.0026]
Joint 6	0	0.11655	0	Link 6	0.615	[0, 0, -0.026]	[0.0000, 0.0000, -0.0000, 0.0000, 0.0004, 0.0000, -0.0000, 0.0000, 0.0003]

Things to Know First

Only need four variables per link

- First rotation in the *x* by α_{i-1}
- Then translation in the *x* by a_{i-1}
- Then rotate in the *z* by θ_i
- Finally translate in the *z* by d_i

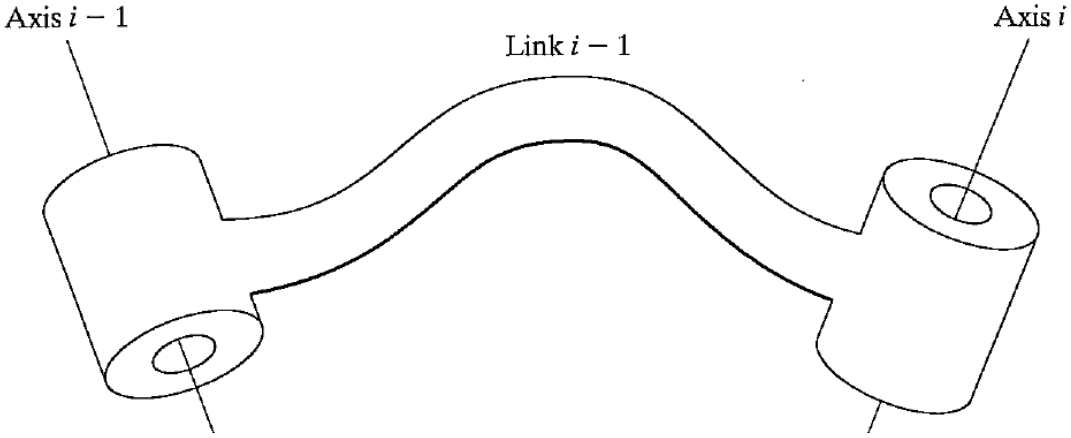
Of course there must be some limitations

- Unfortunately, there are (at least) three different D-H conventions!
- Can only be applied on systems with only revolute or prismatic joints
- It takes some time and practise to get used to the approach

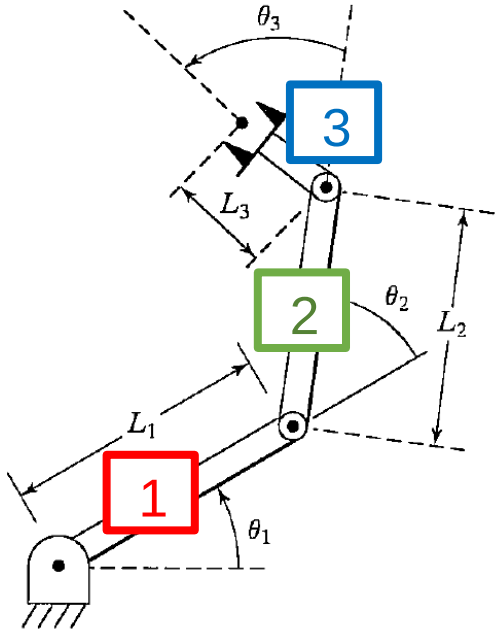
Assigning Frames

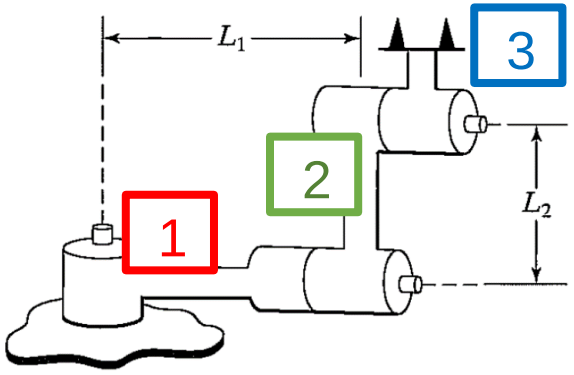
There are three parts needed to learn the D-H method

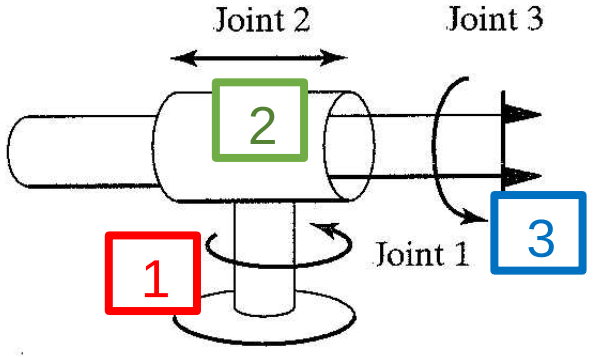
1. Assigning the frames
2. Assigning the variables and the table
3. Compute the transformation matrices (and hence FK)



Assigning Frames: Link Numbers

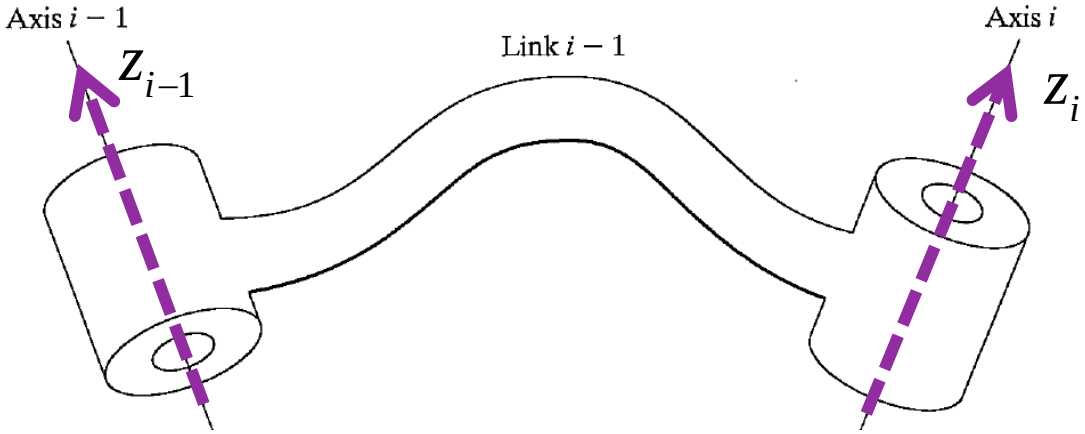






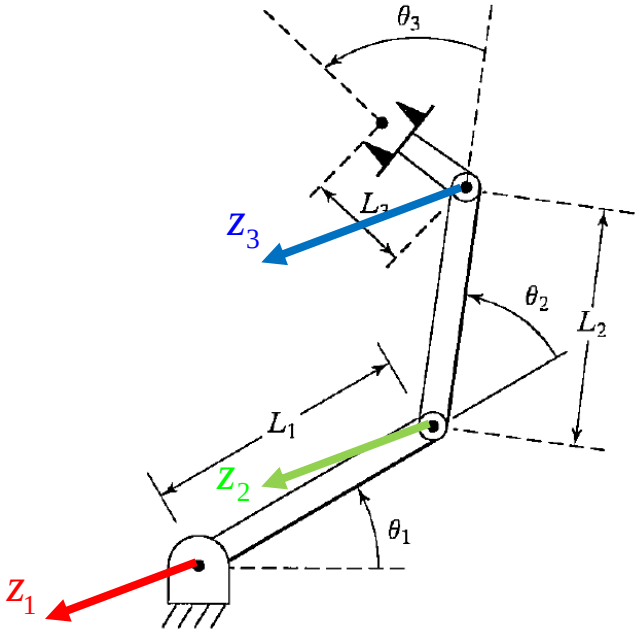
Assigning Frames: Z axes

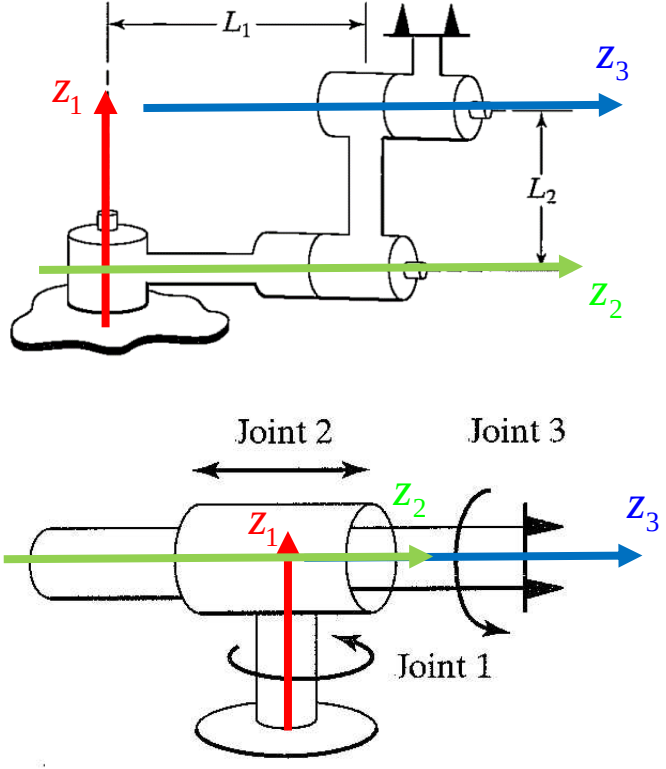
Axis z_i aligns with the axis of rotation/translation for link i



- For the inertial (link 0) axis, simple place it on top of z_1 for convenience

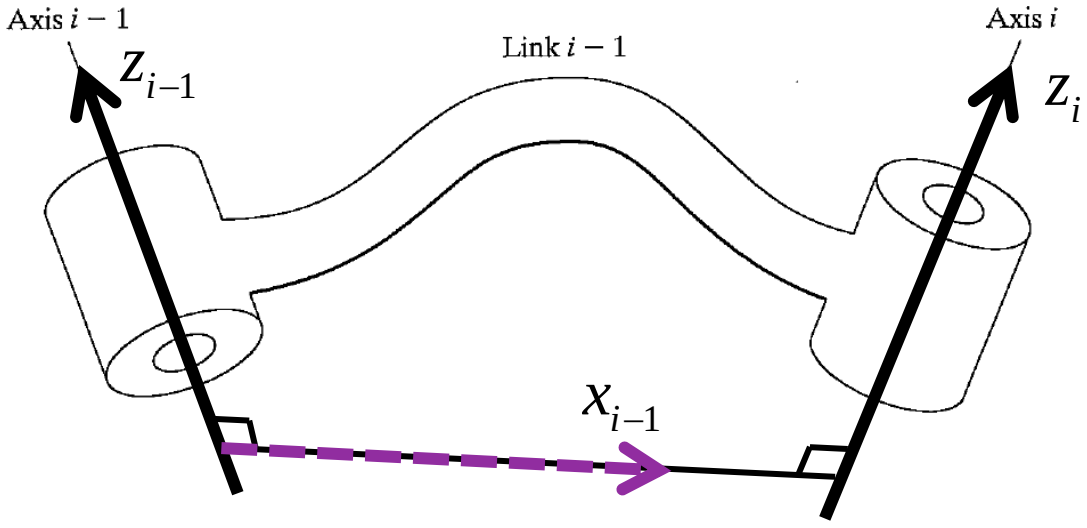
Assigning Frames: Z axes





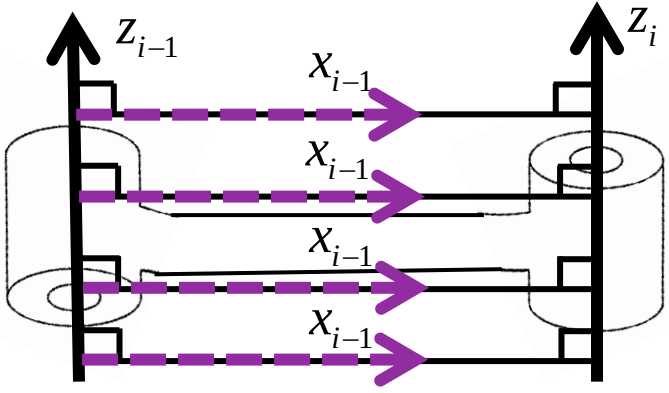
Assigning Frames: X axes

Axis x_{i-1} is the shortest vector between z-axes z_{i-1} and z_i

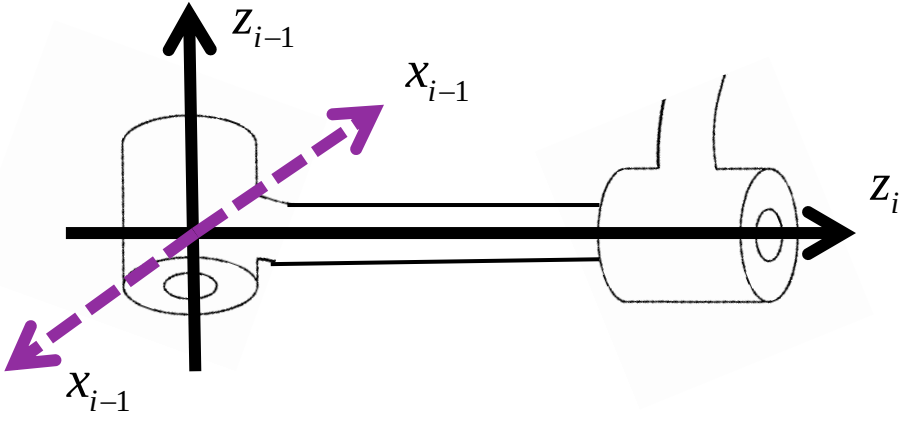


- Convention: axis x_{i-1} points towards axis z_i

Assigning Frames: X axes Special Cases

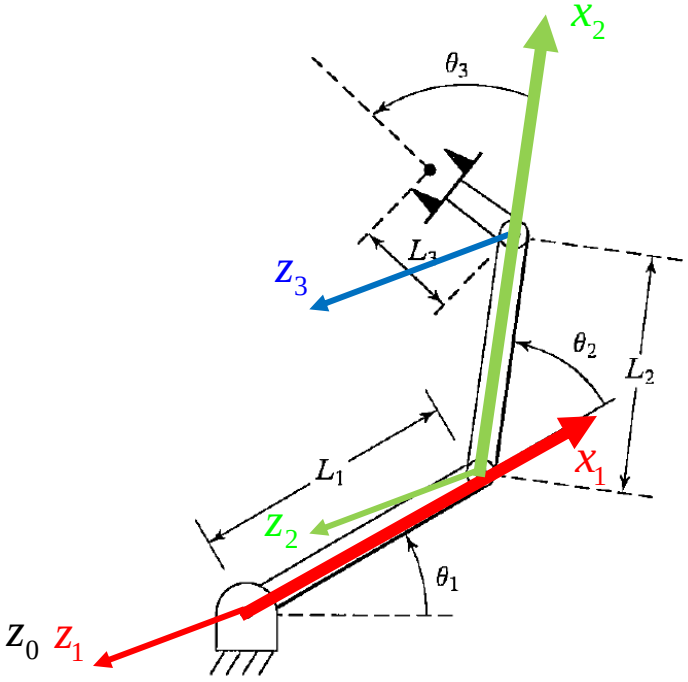


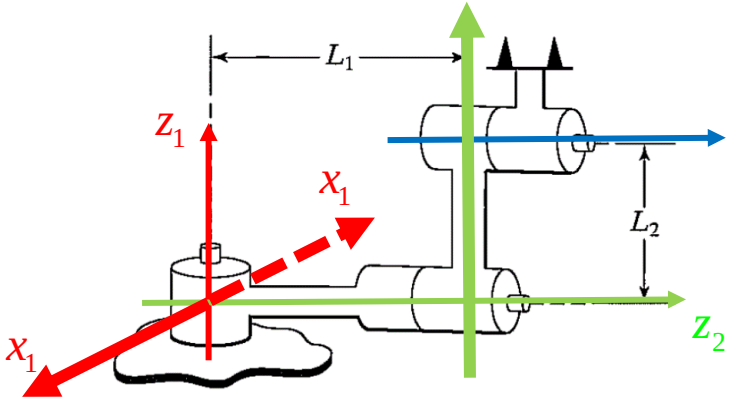
Z-axes parallel

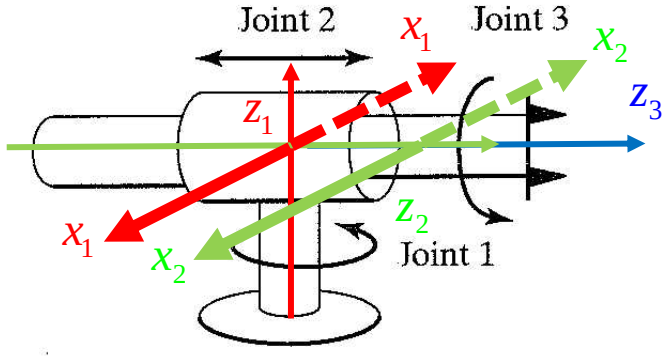


Z-axes intersect

Assigning Frames: X axes



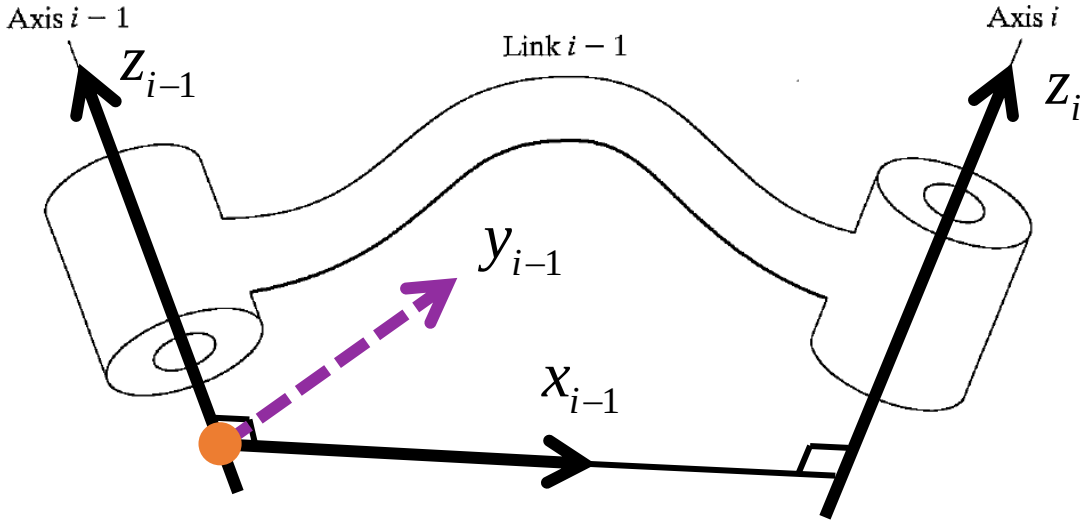




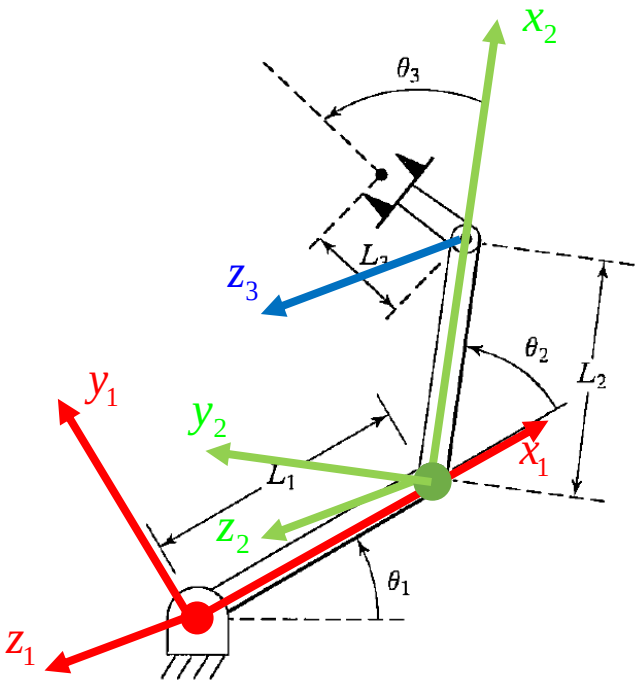
Assigning Frames: The Rest of the Frame

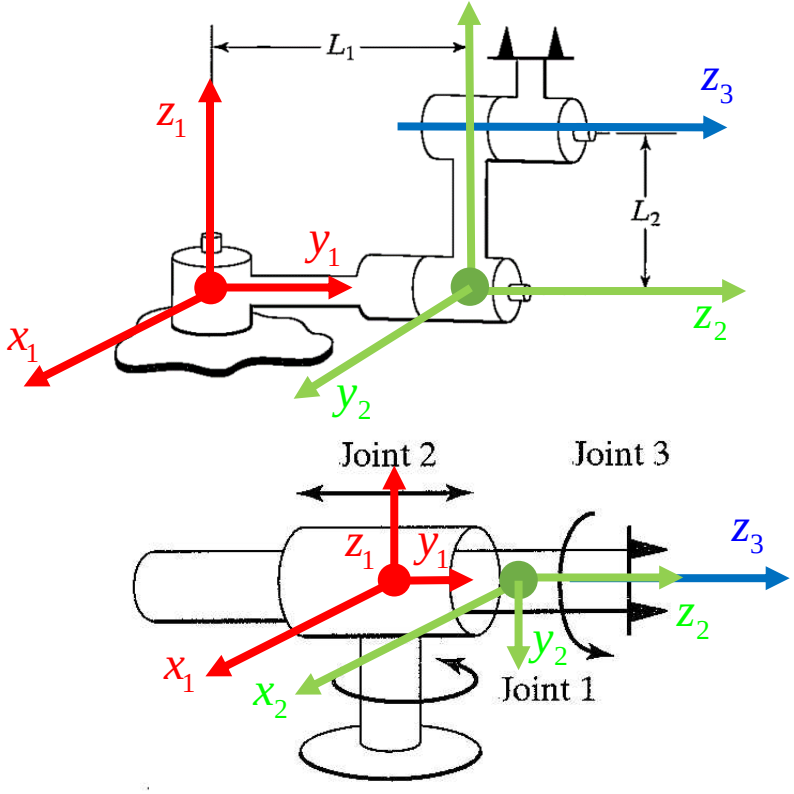
What's left

- Origin of the frame is the intersection of *x* and *z*
- The y axis is naturally assigned by the right hand rule

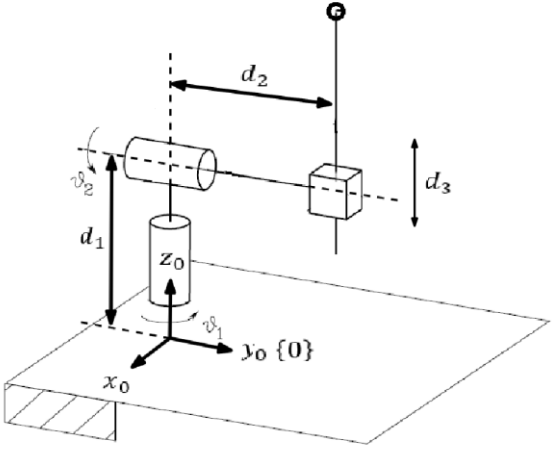


Assigning Frames: The Rest of the Frame

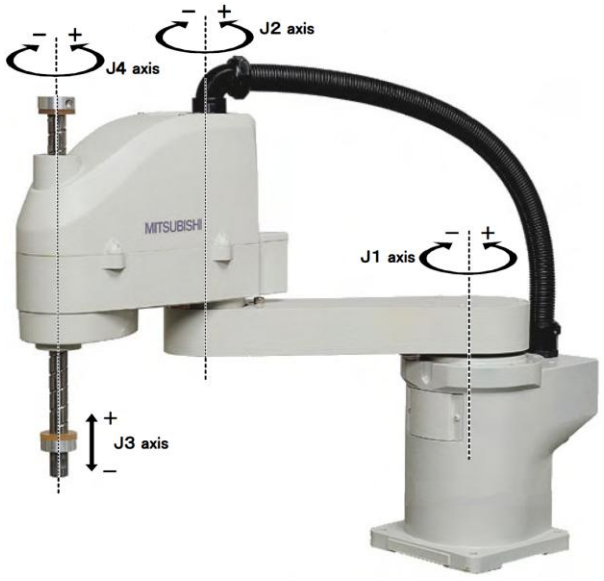




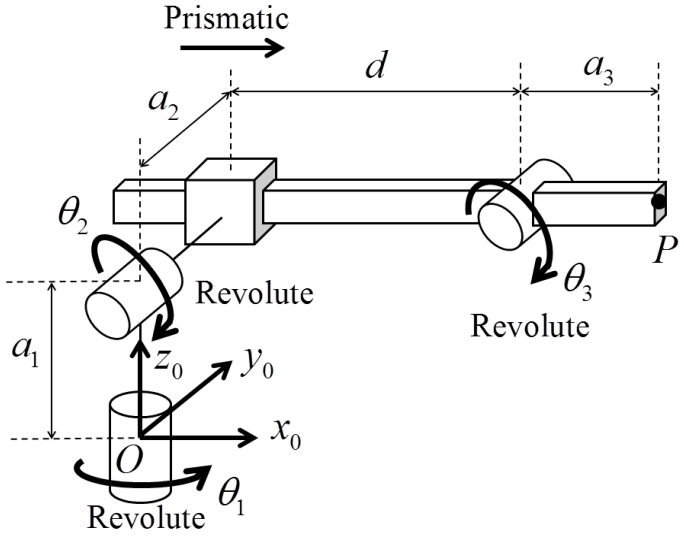
Exercise 1



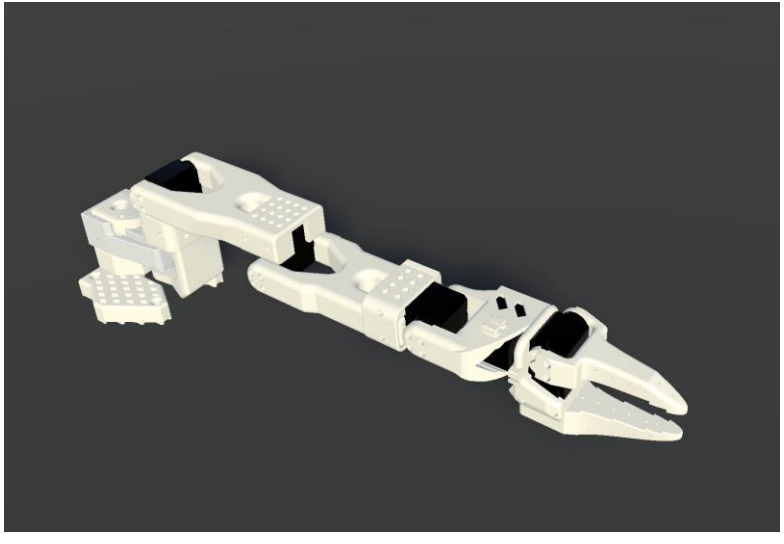
Exercise 2

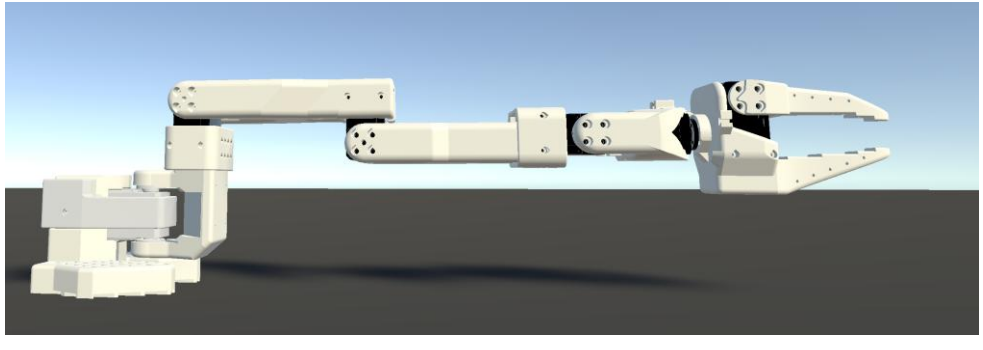


Exercise 3


Prismatic
d
a_3
a_2
θ_2
P
Revolute
θ_3
Revolute
a_1
z_0
y_0
x_0
O
θ_1
Revolute

Exercise 4

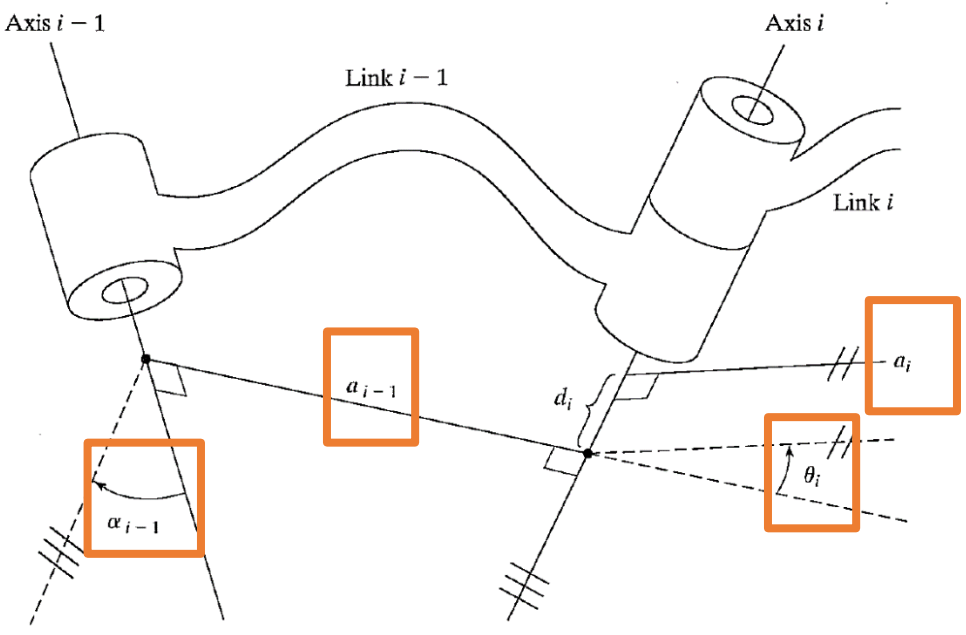




Variables and Table

Determine the D-H parameters

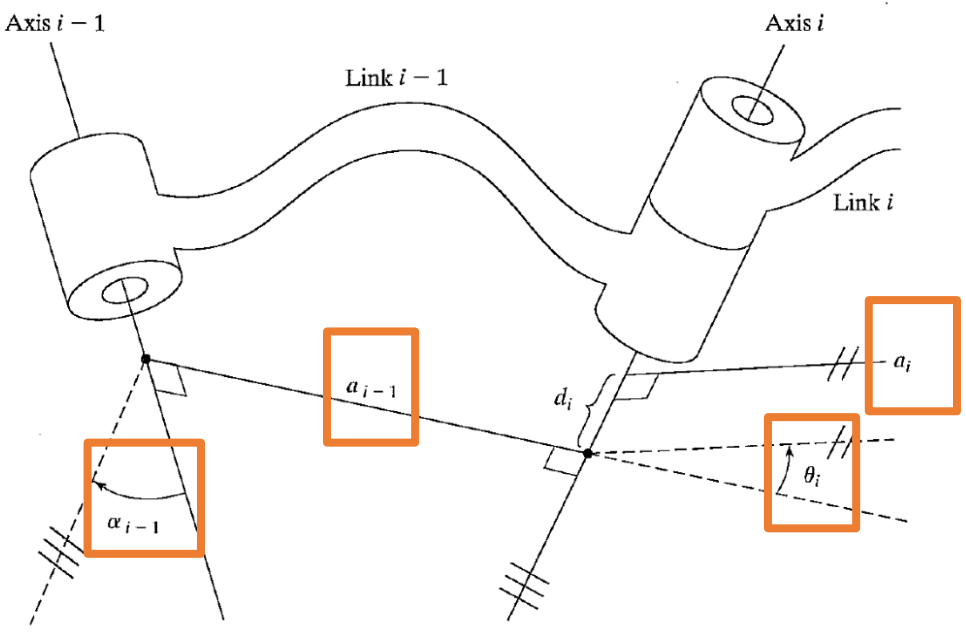
- The **4 parameters** represent the transformation between the frames



Variables and Table

Determine the D-H parameters

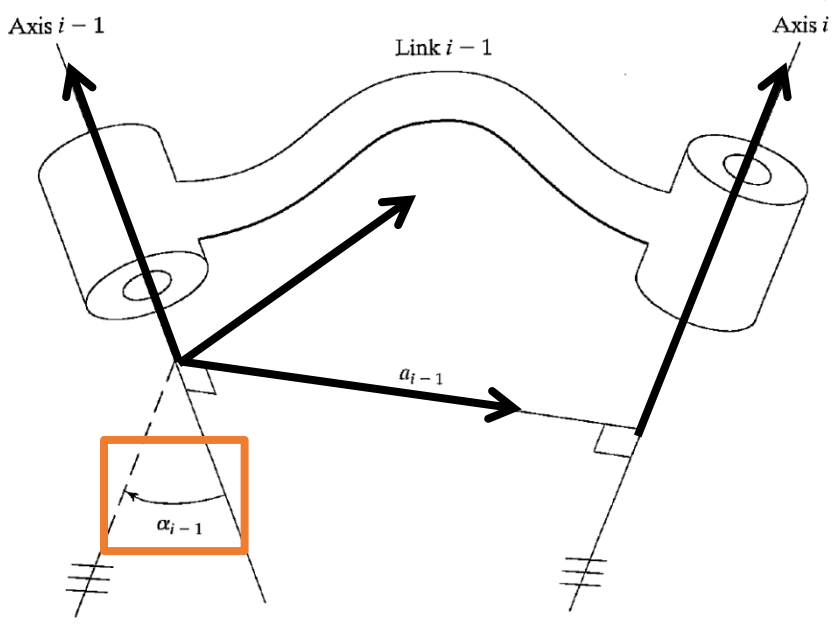
- The **4 parameters** represent the transformation between the frames



First Parameter α_{i-1}

The **rotation** from z_{i-1} to z_i about the **axis x_{i-1}**

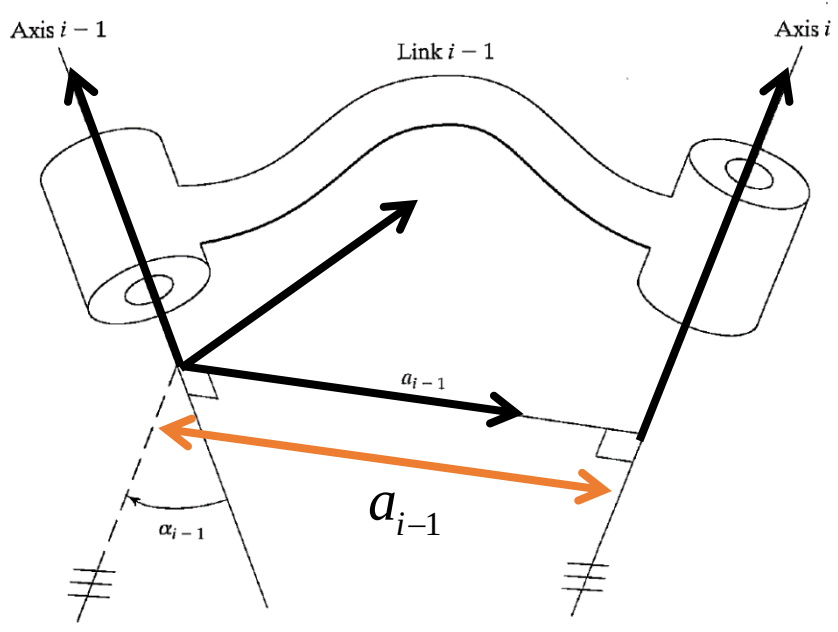
- How to get from the previous *z* axis to this *z* axis



Second Parameter a_{i-1}

The **distance** from z_{i-1} to z_i along the **axis x_{i-1}**

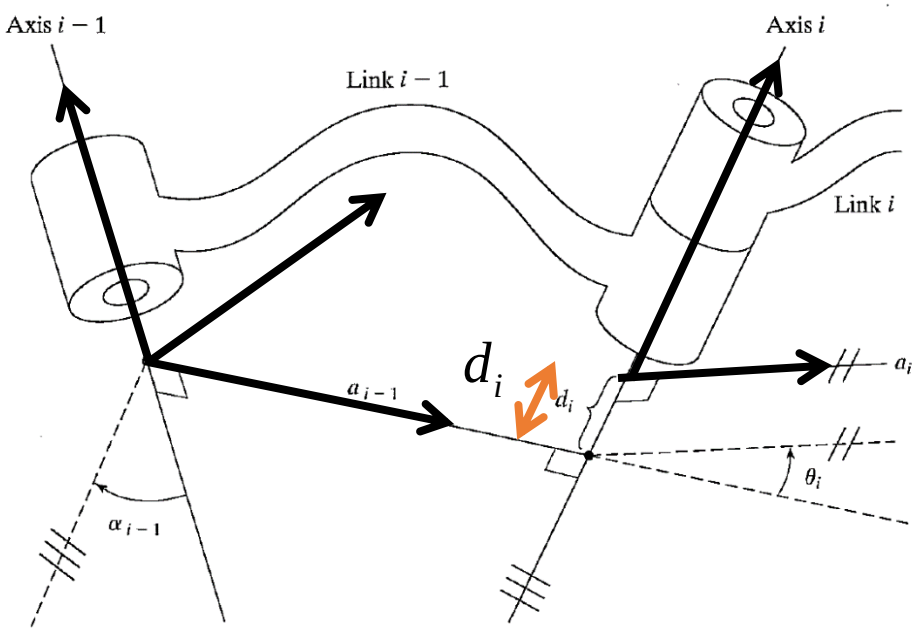
- How to get from the previous z axis to this z axis



Third Parameter d_{i-1}

The **distance** from x_{i-1} to x_i along the **axis z_i**

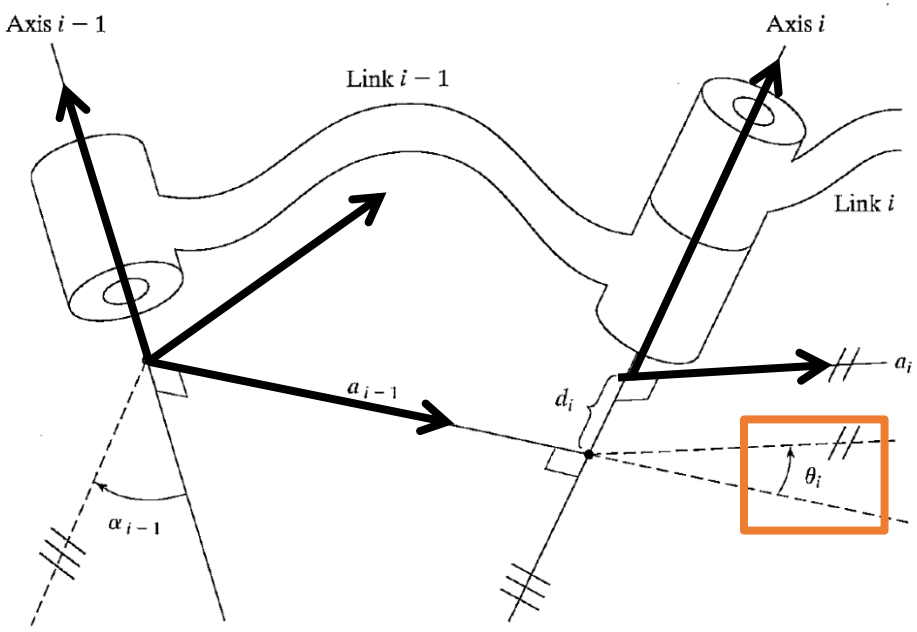
- How to get from the previous x axis to this x axis



Fourth Parameter θ_{i-1}

The **angle** from x_{i-1} to x_i about the **axis z_i**

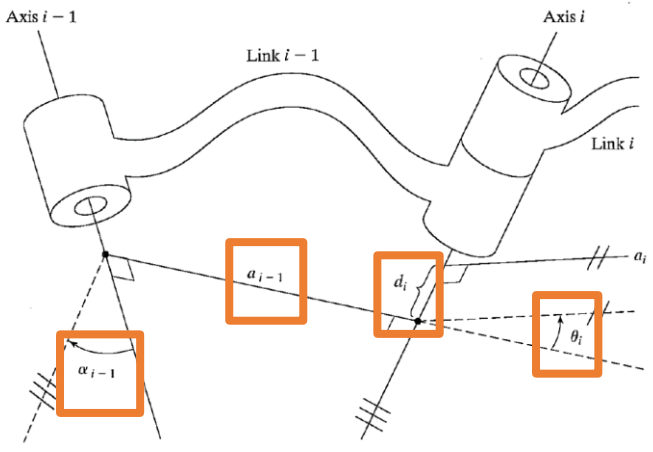
- How to get from the previous x axis to this x axis



Summary of Parameters

The four parameters again

- α_{i-1} : The **angle** from z_{i-1} to z_i about the **axis x_{i-1}**
- a_{i-1} : The **distance** from z_{i-1} to z_i along the **axis x_{i-1}**
- d_i : The **distance** from x_{i-1} to x_i along the **axis z_i**
- θ_i : The **angle** from x_{i-1} to x_i about the **axis z_i**



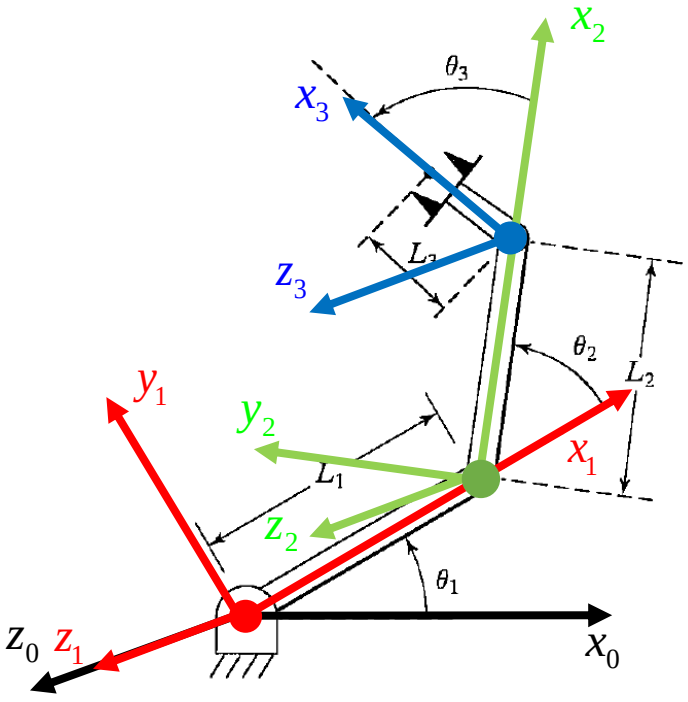
First and Last Frames

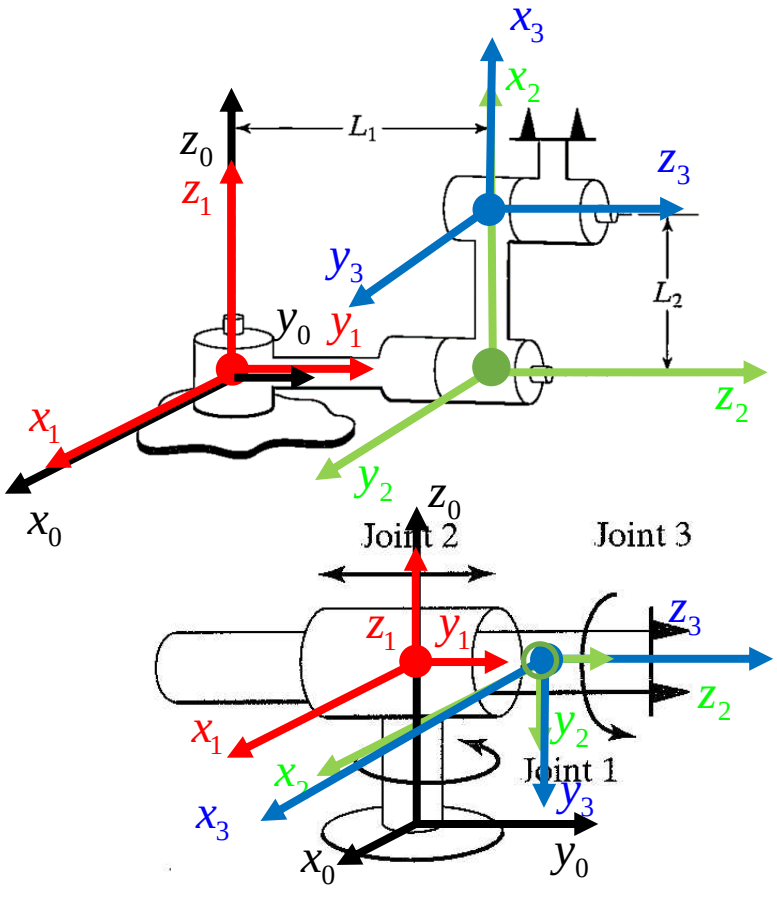
First frame is “sort of” arbitrary (keep it simple)

Last frame

- For revolute joint X_N is chosen to be aligned to X_{N-1} when $\theta_N = 0$, origin of frame is chosen so that $d_N = 0$
- For prismatic joint X_N is chosen such that $\theta_N = 0$, and the origin of frame is chosen at the intersection of X_{N-1} and joint axis N when $d_N = 0$

First and Last Frames





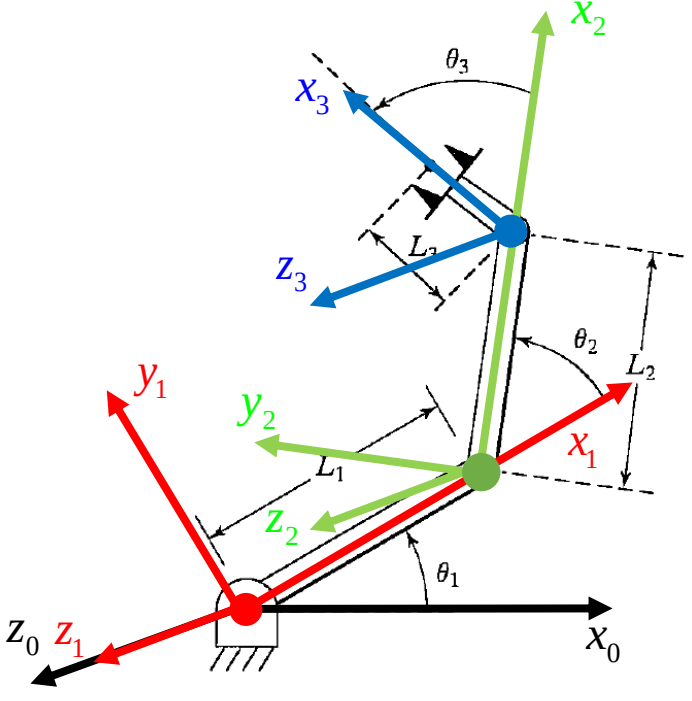
Example Tables

The four parameters again

- α_{i-1} : The **rotation** from z_{i-1} to z_i about the **axis x_{i-1}**
- a_{i-1} : The **distance** from z_{i-1} to z_i along the **axis x_{i-1}**
- d_i : The **distance** from x_{i-1} to x_i along the **axis z_i**
- θ_i : The **angle** from x_{i-1} to x_i about the **axis z_i**

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	0	θ_1
2	0	L_1	0	θ_2
3	0	L_2	0	θ_3

$$\begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & a_{i-1} \\ s_{\theta_i}c_{\alpha_{i-1}} & c_{\theta_i}c_{\alpha_{i-1}} & -s_{\alpha_{i-1}} & -s_{\alpha_{i-1}}d_i \\ s_{\theta_i}s_{\alpha_{i-1}} & c_{\theta_i}s_{\alpha_{i-1}} & c_{\alpha_{i-1}} & c_{\alpha_{i-1}}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



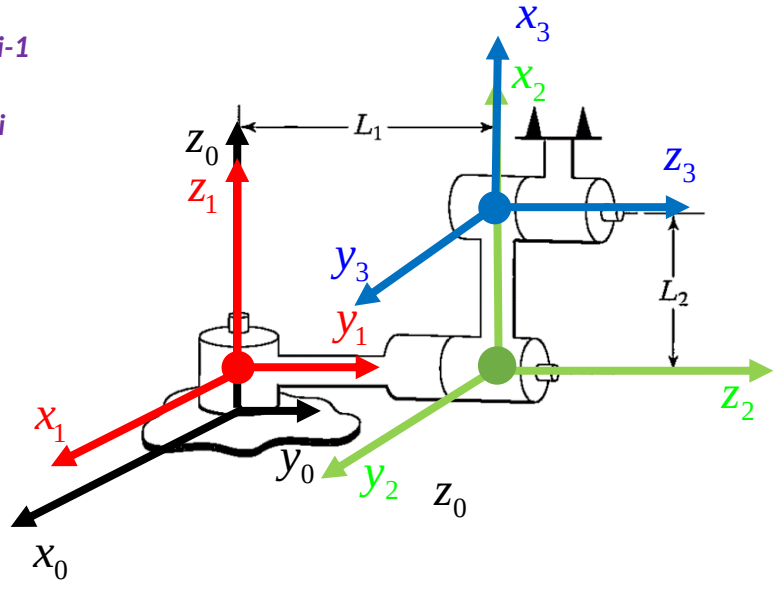
Example Tables

$$\begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & a_{i-1} \\ s_{\theta_i}c_{\alpha_{i-1}} & c_{\theta_i}c_{\alpha_{i-1}} & -s_{\alpha_{i-1}} & -s_{\alpha_{i-1}}d_i \\ s_{\theta_i}s_{\alpha_{i-1}} & c_{\theta_i}s_{\alpha_{i-1}} & c_{\alpha_{i-1}} & c_{\alpha_{i-1}}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The four parameters again

- α_{i-1} : The **rotation** from z_{i-1} to z_i about the **axis x_{i-1}**
- a_{i-1} : The **distance** from z_{i-1} to z_i along the **axis x_{i-1}**
- d_i : The **distance** from x_{i-1} to x_i along the **axis z_i**
- θ_i : The **angle** from x_{i-1} to x_i about the **axis z_i**

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	L_0	θ_1
2	-90°	0	L_1	$-90+\theta_2$
3	0	L_2	0	θ_3

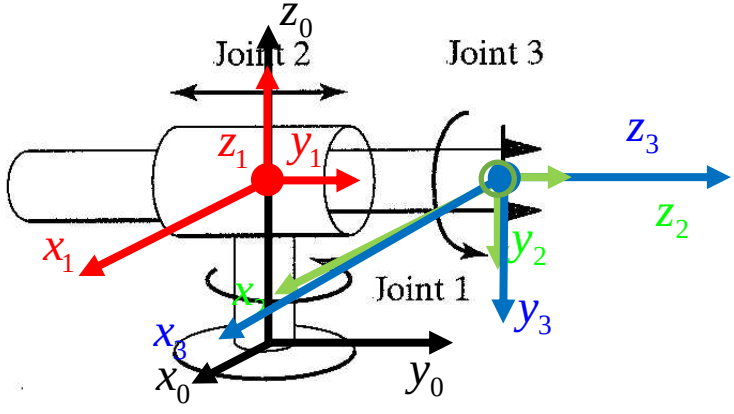


Example Tables

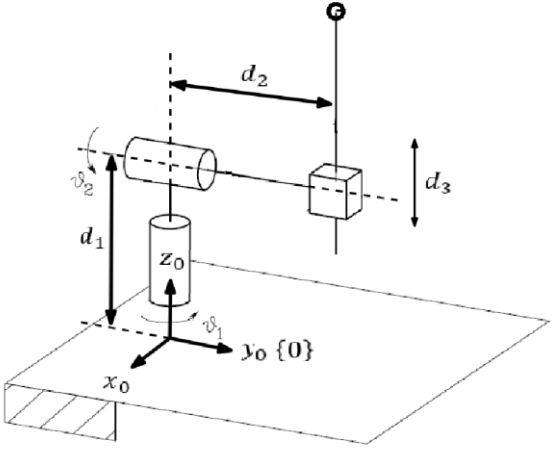
The four parameters again

- α_{i-1} : The **rotation** from z_{i-1} to z_i about the **axis x_{i-1}**
- a_{i-1} : The **distance** from z_{i-1} to z_i along the **axis x_{i-1}**
- d_i : The **distance** from x_{i-1} to x_i along the **axis z_i**
- θ_i : The **angle** from x_{i-1} to x_i about the **axis z_i**

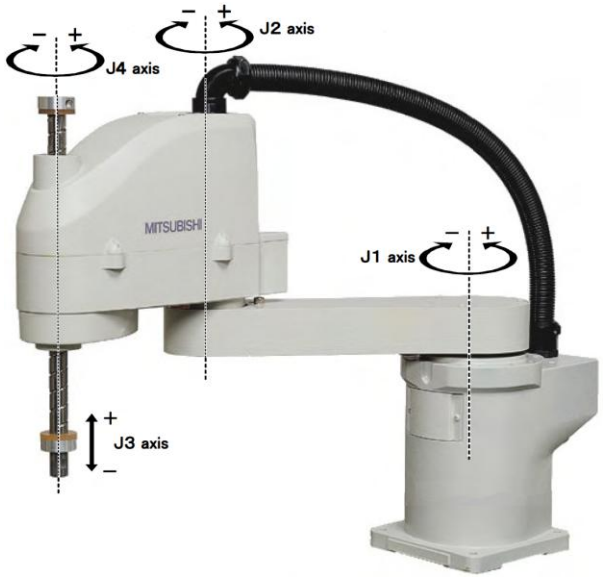
i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	L_1	θ_1
2	-90°	0	d	0
3	0	0	0	θ_2



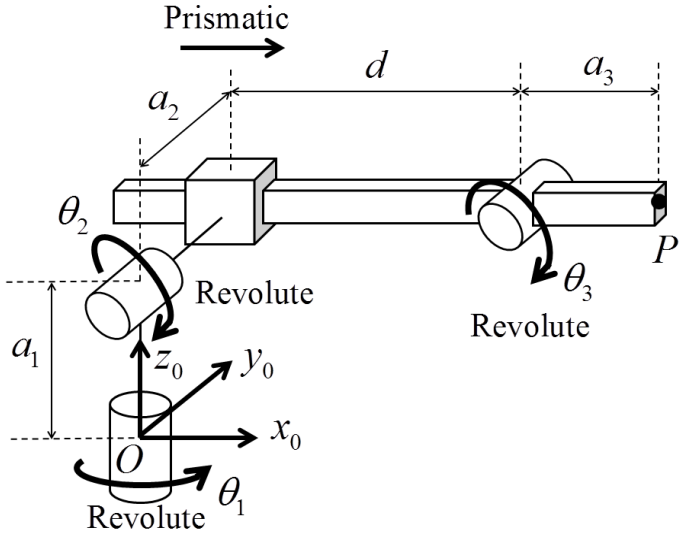
Exercise 1



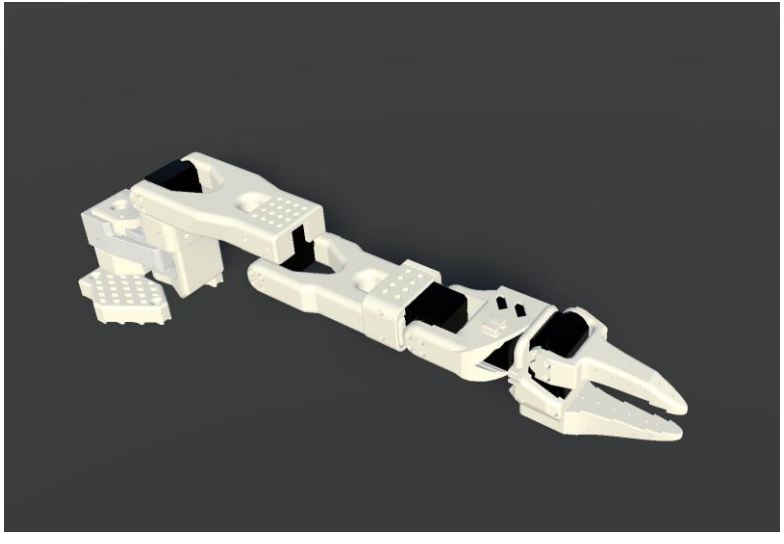
Exercise 2

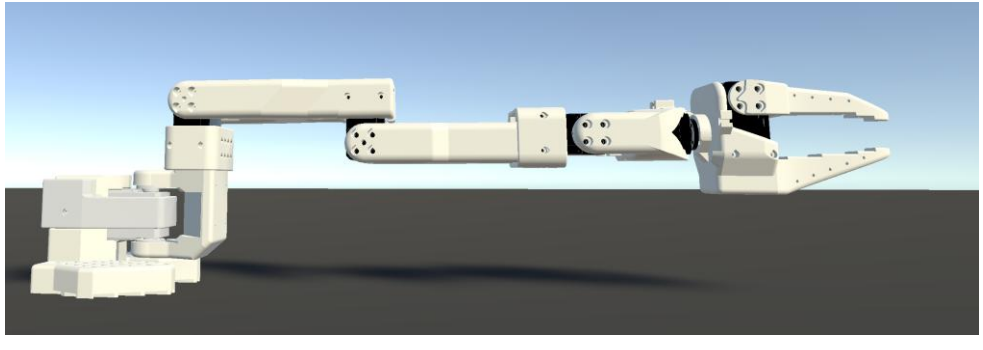


Exercise 3

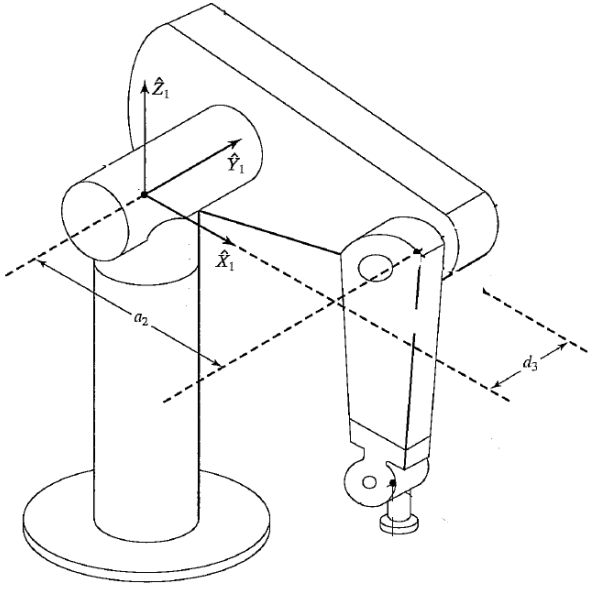

Prismatic
d
a_3
a_2
θ_2
Revolute
θ_3
Revolute
P
a_1
z_0
y_0
x_0
O
θ_1
Revolute

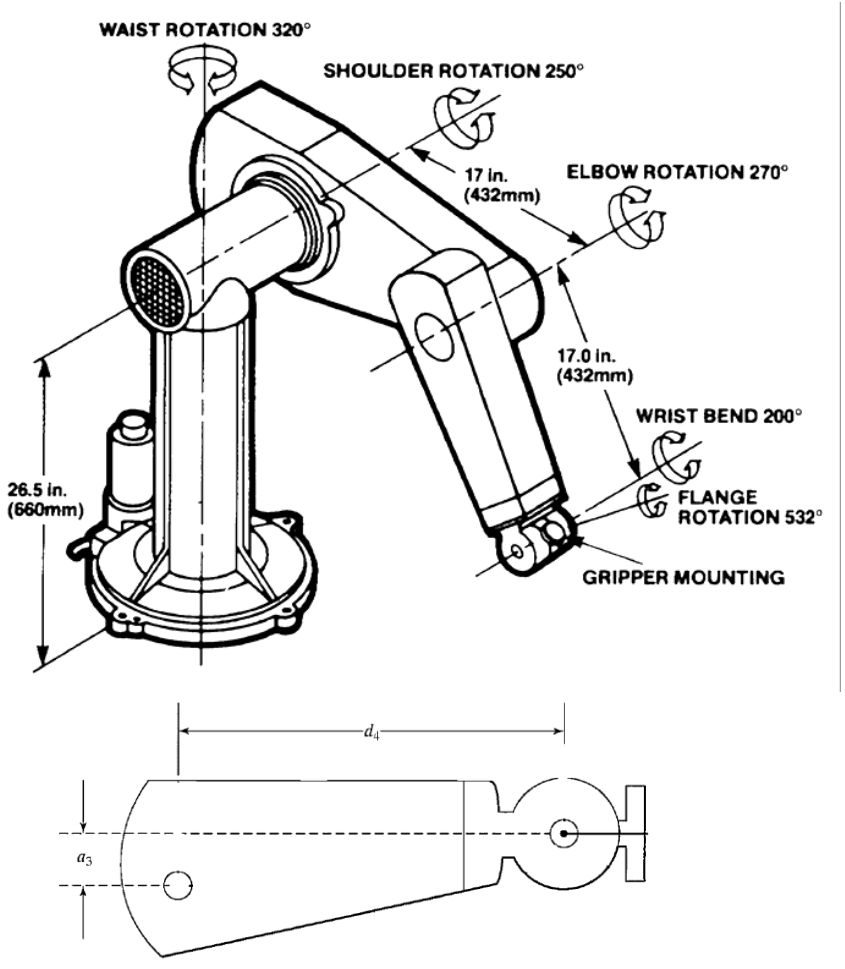
Exercise 4





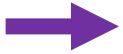
Exercise 5


$\hat{Z}_1$
$\hat{Y}_1$
$\hat{X}_1$
a_2
d_3


WAIST ROTATION 320°
SHOULDER ROTATION 250°
17 in.
(432mm)
ELBOW ROTATION 270°
17.0 in.
(432mm)
WRIST BEND 200°
26.5 in.
(660mm)
FLANGE
ROTATION 532°
GRIPPER MOUNTING
d_4
a_3

Exercise 5

i	α_{i-1}	a_{i-1}	d_i	θ_i
1				
2				
3				
4				
5				
6				



i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	0	θ_1
2	-90°	0	0	θ_2
3	0	a_2	d_3	θ_3
4	-90°	a_3	d_4	θ_4
5	90°	0	0	θ_5
6	-90°	0	0	θ_6

D-H Transformation Matrix

Can be expressed by the homogeneous transformation

$$
{}^{i-1}_{i}T = R_x(\alpha_{i-1})D_x(a_{i-1})R_z(\theta_i)D_z(d_i)
$$

$$
= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_{i-1}} & -s_{\alpha_{i-1}} & 0 \\ 0 & s_{\alpha_{i-1}} & c_{\alpha_{i-1}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_{i-1} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & a_{i-1} \\ s_{\theta_i}c_{\alpha_{i-1}} & c_{\theta_i}c_{\alpha_{i-1}} & -s_{\alpha_{i-1}} & -s_{\alpha_{i-1}}d_i \\ s_{\theta_i}s_{\alpha_{i-1}} & c_{\theta_i}s_{\alpha_{i-1}} & c_{\alpha_{i-1}} & c_{\alpha_{i-1}}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

Rotation matrix

Translation

Why Is The Definition Different Online?

What I learnt is wrong!?!

$$
{}^{n-1}T_n = \left[\begin{array}{ccc|c}
\cos\theta_n & -\sin\theta_n\cos\alpha_n & \sin\theta_n\sin\alpha_n & r_n\cos\theta_n \\
\sin\theta_n & \cos\theta_n\cos\alpha_n & -\cos\theta_n\sin\alpha_n & r_n\sin\theta_n \\
0 & \sin\alpha_n & \cos\alpha_n & d_n \\
\hline
0 & 0 & 0 & 1
\end{array}\right]
$$

$$
{}^{n-1}T_n = \left[\begin{array}{ccc|c}
\cos\theta_n & -\sin\theta_n & 0 & a_{n-1} \\
\sin\theta_n\cos\alpha_{n-1} & \cos\theta_n\cos\alpha_{n-1} & -\sin\alpha_{n-1} & -d_n\sin\alpha_{n-1} \\
\sin\theta_n\sin\alpha_{n-1} & \cos\theta_n\sin\alpha_{n-1} & \cos\alpha_{n-1} & d_n\cos\alpha_{n-1} \\
\hline
0 & 0 & 0 & 1
\end{array}\right]
$$

$$
{}^{n-1}T_n = \mathrm{Rot}_{z_{n-1}}(\theta_n)\cdot\mathrm{Trans}_{z_{n-1}}(d_n)\cdot\mathrm{Trans}_{x_n}(r_n)\cdot\mathrm{Rot}_{x_n}(\alpha_n)
$$

$$
{}^{n-1}T_n = \mathrm{Rot}_{x_{n-1}}(\alpha_{n-1})\cdot\mathrm{Trans}_{x_{n-1}}(a_{n-1})\cdot\mathrm{Rot}_{z_n}(\theta_n)\cdot\mathrm{Trans}_{z_n}(d_n)
$$

Final Forward Kinematics

Finally, the forward kinematics can be determined by computing the transformation matrix from the end-effector to the base frame

$$ {}^{0}_{N}T = {}^{0}_{1}T\ {}^{1}_{2}T\ {}^{2}_{3}T \cdots {}^{N-1}_{N}T $$

- The meaning of the transformation matrix is

$$ {}^{0}_{N}T = \left[\begin{array}{ccc|c} & {}^{0}_{N}R & & {}^{0}\mathbf{r}_{NORG} \\ \hline 0 & 0 & 0 & 1 \end{array}\right] $$

- Containing the **orientation** and **translation** to the origin of the end-frame, and finally

$$ {}^{0}\mathbf{r}_{EE} = {}^{0}_{N}T\ {}^{N}\mathbf{r}_{EE} $$

Analytical vs Numerical

Analytical (or closed form) means

- There is an algebraic expression for the final answer

$$\begin{bmatrix} P_x \\ P_y \end{bmatrix} = \begin{bmatrix} l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2) \\ l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2) \end{bmatrix}$$

- Advantages include: **fast to compute** and **understanding of the mechanism**

Numerical means

- Directly substitute just to get the "number" answer

$$\begin{bmatrix} P_x \\ P_y \end{bmatrix} = {}^0_1T(\theta_1 = 30°)\,{}^1_2T(\theta_1 = 60°) = \begin{bmatrix} 8.6603 \\ 15 \end{bmatrix} \qquad l_1 = l_2 = 10$$

- Advantages include: **easy to scale to any type of robot**

Analytical Forward Kinematics

So how to calculate the closed-form kinematics, it is not hard, either

- Compute it by hand

$$ {}^{0}_{1}T = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 & 0 \\ \sin\theta_1 & \cos\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{1}_{2}T = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & l_1 \\ \sin\theta_2 & \cos\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{2}_{3}T = \begin{bmatrix} 1 & 0 & 0 & l_2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

- Compute it using software

```
>>> import sympy
>>> a1, a2 = sympy.symbols("a1 a2")
>>> e = ET2.R() * ET2.tx(a1) * ET2.R() * ET2.tx(a2);
>>> q0, q1 = sympy.symbols("q0 q1")
>>> TE = e.fkine([q0, q1])
cos(q0 + q1)   -sin(q0 + q1)   a1*cos(q0) + a2*cos(q0 + q1)
sin(q0 + q1)    cos(q0 + q1)   a1*sin(q0) + a2*sin(q0 + q1)
  0             0              1
```

Numerical Forward Kinematics

Transformations (and DH tables) really helps

- A “for” loop would solve it!

for *i = 1* **to** *N* **do**
$^{i-1}\boldsymbol{T}_i$ = transform($\boldsymbol{q}_i$)
$^{0}\boldsymbol{T}_i = {}^{0}\boldsymbol{T}_{i-1}\ {}^{i-1}\boldsymbol{T}_i$
end

- With initial condition

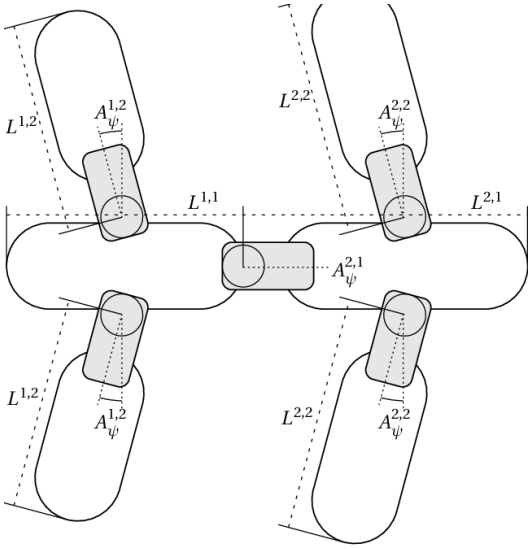
$$ {}^{0}_{0}T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Branched Mechanisms

Serial robots can be easily extended to branched mechanisms

- Parent of link *i*: $\lambda(i)$
- Child of link *i*: $\mu_i = \{ j : i = \lambda(j) \}$

- Following a naming convention of from 1 (distal) to *N*



The FK algorithm would become

for *i = 1* **to** *N* **do**
  ${}^{\lambda(i)}\boldsymbol{T}_i$ = transform($\boldsymbol{q}_i$)
  ${}^{0}\boldsymbol{T}_i = {}^{0}\boldsymbol{T}_{\lambda(i)}\ {}^{\lambda(i)}\boldsymbol{T}_i$
end

Unified Robot Description Format (URDF)

XML format for representing a robot model

- It's actually very natural

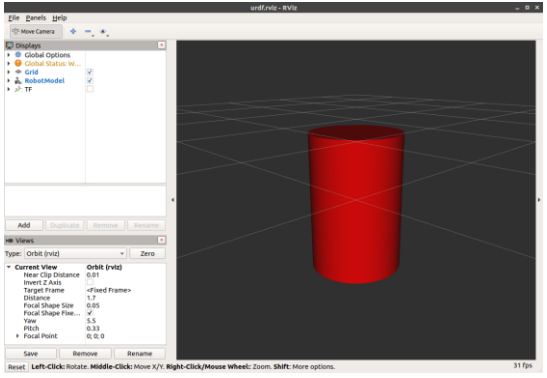
```
<?xml version="1.0"?>
<robot name="myfirst">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </visual>
  </link>
</robot>
```

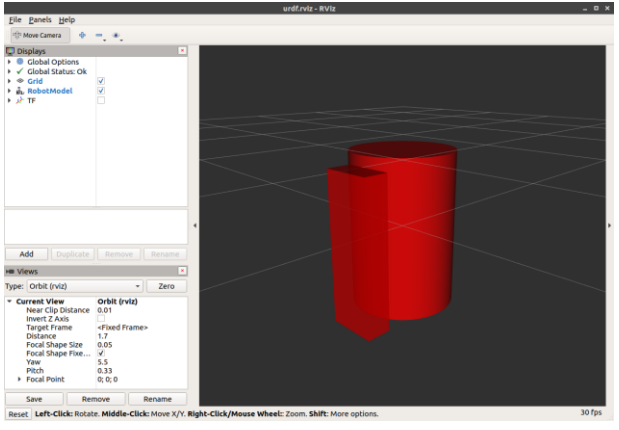
```
<?xml version="1.0"?>
<robot name="origins">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </visual>
  </link>

  <link name="right_leg">
    <visual>
      <geometry>
        <box size="0.6 0.1 0.2"/>
      </geometry>
      <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    </visual>
  </link>

  <joint name="base_to_right_leg" type="fixed">
    <parent link="base_link"/>
    <child link="right_leg"/>
    <origin xyz="0 -0.22 0.25"/>
  </joint>

</robot>
```





Unified Robot Description Format (URDF)

Adding joints and constraints

```
<joint name="head_swivel" type="continuous">
  <parent link="base_link"/>
  <child link="head"/>
  <axis xyz="0 0 1"/>
  <origin xyz="0 0 0.3"/>
</joint>
```

```
<joint name="left_gripper_joint" type="revolute">
  <axis xyz="0 0 1"/>
  <limit effort="1000.0" lower="0.0" upper="0.548" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0.2 0.01 0"/>
  <parent link="gripper_pole"/>
  <child link="left_gripper"/>
</joint>
```

```
<joint name="gripper_extension" type="prismatic">
  <parent link="base_link"/>
  <child link="gripper_pole"/>
  <limit effort="1000.0" lower="-0.38" upper="0" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0.19 0 0.2"/>
</joint>
```

Joints

```
<link name="base_link">
  <visual>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 .8 1"/>
    </material>
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="10"/>
    <inertia ixx="1e-3" ixy="0.0" ixz="0.0" iyy="1e-3" iyz="0.0" izz="1e-3"/>
  </inertial>
</link>
```

Geometry and collision